

LC 2603: Collect Coins in a Tree

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 31, 2026

Uber • Google • Cisco

Rules & Problem Statement

Rule 1: Khade khade apne se next node ya phir apne se next ke next node se coin utha sakte.

Rule 2: Ek vertex change karne ke liye cost++.

Rule 3: Can start from any vertex, need to return min cost of traversal.

Difference from Collect Apple Problem:

This is different from the *Collect Apple* problem because there we needed to physically visit the node having the apple, and if a subtree had an apple then we did cost += 2 (aana aur jaana).

Here, the same idea won't work because even a distance of 2 is allowed. No need to physically visit the node.

Core Approach (Topological / Kahn's Algo)

Step 1 (S1): Pruning Useless Leaves

Remove every leaf having NO coin, eventually.

Eventually all the leaf nodes with no coin will leave, and we only have leaf nodes with coins left in the tree.

Step 2 (S2): Removing Layers

Now remove 2 layers from the remaining leaves. Whatever is left is the walkable part.

Visual Case Intuition

If a node has no coin, it will get removed. It will decrease the degree of its neighbors, and if a neighbor's degree becomes 1 and it also has no coin, it is also eligible to get removed. (Topological Kahn's Algorithm Approach).

C++ Solution

```

class Solution {
public:
    int collectTheCoins(vector<int>& coins, vector<vector<int>>&
edges) {
        // basic crux ye he ki
        // har ek wo leaf node jisme coin nahi wo useless he
        // to waise leaf node ko hatate jaayenge

        // pruning ke baad har leaf node pe coin hoga
        // uske baad 2 layer remove kr denge
        // bacche hue me travel kr lenge
        int n = coins.size();
        vector<vector<int>> adj(n);
        vector<int> degree(n, 0);

        for(auto &it : edges) {
            int u = it[0];
            int v = it[1];

            adj[u].push_back(v);
            adj[v].push_back(u);
            degree[u]++;
            degree[v]++;
        }

        queue<int> q;
        for(int i = 0; i < n; i++) {
            if(degree[i] == 1 && coins[i] == 0) {
                // useless leaf node
                q.push(i); // queue me isliye kyunki ho skta iske
// iske removal ke baad ek useless leaf
// saare leaf node pe coin hoga
            }
        }

        while(!q.empty()) {
            int curr = q.front();
            q.pop();
            degree[curr]--;

            for(auto &it : adj[curr]) {
                if(degree[it] == 0) continue; // already removed
                degree[it]--;
                if(degree[it] == 1 && coins[it] == 0) {
                    q.push(it);
                }
            }
        }

        // pruning ke baad ab
        // saare leaf node pe coin hoga
    }
};

```

```

// 2 layer remove krna hoga
// 1 leaf node waala layer , aur ek uske upar waala
for(int i = 0; i < n; i++) {
    if(degree[i] == 1) {
        q.push(i);
    }
}

for(int layer = 0; layer <= 1; layer++) { // bas 2 hi baar
    queue level prune krenge
    int currsz = q.size();
    while(currsz--) {
        int curr = q.front();
        q.pop();
        degree[curr]--;

        for(auto &it : adj[curr]) {
            if(degree[it] == 0) continue;
            degree[it]--;
            if(degree[it] == 1) {
                q.push(it);
            }
        }
    }
}

int rem = 0;
for(int i = 0; i < n; i++) {
    rem += degree[i];
}

rem /= 2; // ek hi edge do baar count hota
rem *= 2; // ek baar aana ek baar jaana
return rem;
}
};

```